# Molecular Properties Prediction with Graph Neural Network

Chanop Sethiwan (e1351107@u.nus.edu), A0296969W
Neo Jun Wei (e1351841@u.nus.edu), A0297703W
Group 30
July 27, 2025

## 1 Introduction

The prediction of molecular properties, particularly their interactions with critical biological targets such as the androgen receptor, is a fundamental task in computational drug discovery and toxicology. The androgen receptor, a type of nuclear hormone receptor, plays a key role in physiological processes by binding to hormones like testosterone. Accurately predicting whether a molecule exhibits biological activity toward such receptors is essential for identifying potential drug candidates with desirable therapeutic effects and minimal side effects. In this study, activity is determined based on binary assay outcomes (active or inactive) provided by the Tox21 dataset.

This study utilises the **Tox21** dataset, a well-established benchmark in computational toxicology, containing data on over 8,000 molecules and their activity across 12 different nuclear receptor and stress response pathways. Molecules in this dataset are represented using the **SMILES** (Simplified Molecular Input Line Entry System) notation, which encodes chemical structures as text, enabling efficient storage and computational processing.

To predict molecular interactions with the 12 targets in Tox21, we explore two principal approaches:

1. **Extended Connectivity Fingerprints (ECFP4):** SMILES strings are transformed into ECFP4 fingerprints, which encode the local atomic environment within a fixed radius. These fingerprints serve as high-dimensional feature vectors suitable for input into traditional machine learning models for binary classification tasks.

2. **Graph-Based Representations:** Alternatively, SMILES strings can be converted into molecular graphs, where atoms are treated as nodes and chemical bonds as edges. This representation allows for the application of **Graph Neural Networks (GNNs)**, which are capable of learning hierarchical and structural features directly from the graph topology.

In this report, we benchmark and compare traditional machine learning models—Logistic Regression, Support Vector Machine (SVM), Random Forest, and XGBoost—with graph-based approaches, including single-task GCN (st-GCN), multi-task GCN (mt-GCN), and Message Passing Neural Networks (MPNN), to evaluate their effectiveness in predicting compound activity within the Tox21 framework.

## 2 Methodology

This section outlines the methodology adopted in this study, beginning with an exploration of the Tox21 dataset, followed by a presentation of the two primary modeling approaches used to predict compound activity across 12 nuclear receptor and stress response targets.

### 2.1 Initial Dataset Exploration

Before developing predictive models, we conducted a thorough analysis of the Tox21 dataset to understand its structure, identify potential limitations, and guide preprocessing decisions. The dataset comprises molecular activity data for over 8,000 compounds across 12 biological targets.

One of the primary challenges was the presence of missing labels. On average, approximately 17.3% of the activity data is missing, with the proportion varying across targets. Since reliable imputation was not feasible and supervised learning requires complete labels, we adopted a target-wise filtering strategy: for each target-specific model, only samples with non-missing labels for that target were retained. This allowed each model to be trained and evaluated with clean supervision while avoiding the noise introduced by speculative imputations. In the case of multi-task models, we

applied a masking mechanism to ignore missing labels during loss computation, thus preventing NaN propagation.

Another critical characteristic of the dataset is the high degree of class imbalance. Table 1 shows the distribution of active and inactive samples per target, revealing a pronounced skew toward the inactive class.

| **Target** | **Inactive** | **Active** | **Active Proportion** |
|---|---|---|---|
| SR-ARE | 4967 | 961 | 0.162 |
| SR-MMP | 4978 | 936 | 0.158 |
| NR-ER | 5513 | 796 | 0.126 |
| NR-AhR | 5899 | 785 | 0.117 |
| SR-p53 | 6471 | 431 | 0.062 |
| SR-HSE | 6208 | 379 | 0.058 |
| NR-Aromatase | 5627 | 307 | 0.052 |
| NR-ER-LBD | 6750 | 355 | 0.050 |
| NR-AR | 7123 | 309 | 0.042 |
| SR-ATAD5 | 6958 | 267 | 0.037 |
| NR-AR-LBD | 6657 | 238 | 0.035 |
| NR-PPAR-gamma | 6387 | 189 | 0.029 |

Table 1: Distribution of active and inactive samples for each target.

To address this imbalance and ensure robust evaluation, we employed **stratified 5-fold cross-validation** across all modeling tasks. This method maintains the proportion of active and inactive samples in each fold, enabling more representative training and testing splits. Additionally, we adopted the **Area Under the Receiver Operating Characteristic Curve (AUC-ROC)** as the primary evaluation metric. Compared to accuracy, which can be misleading in imbalanced scenarios, AUC-ROC provides a threshold-independent measure of a model's ability to distinguish between positive and negative classes, capturing the trade-off between true positive and false positive rates.

## 2.2 Using ECFP4 with Machine Learning Architectures

To evaluate the predictive potential of classical machine learning methods, we tested four classifiers—Logistic Regression, Support Vector Machine (SVM), Random Forest, and XGBoost—on the Tox21 dataset. Each compound was represented as a fixed-length binary vector using ECFP4 (Extended Connectivity Fingerprints), generated via the RDKit library. Among the chosen models, Random Forest and XGBoost are tree-based ensemble methods, while Logistic Regression and SVM serve as non-tree-based baselines.

### 2.2.1 Hyperparameter Tuning and Evaluation Strategy

All models were trained and evaluated using **stratified 5-fold cross-validation**, consistent with the strategy outlined in the previous subsection. Stratification ensured that the class imbalance was preserved within each fold, reducing variance and enabling reliable comparison across models and targets.

Hyperparameter tuning was performed using `RandomizedSearchCV` within each training fold. The best parameter sets identified were then used to train and validate the models within the cross-validation framework—eliminating the need for a separate held-out test set and minimizing the risk of performance inflation.

Each model's performance was assessed using **ROC-AUC**. These metrics were computed for each fold and averaged across the five folds for each toxicity endpoint. The resulting mean scores were compiled into comparative tables to facilitate both per-endpoint analysis and cross-model benchmarking in the results section.

## 2.3 Using Molecular Graphs with Graph-Based Architectures

In this subsection, we describe the methodology for utilising molecular graph representations in predictive modeling. Unlike fixed-length fingerprint vectors, molecular graphs provide a more expressive and structured form to capture atomic connectivity and spatial relationships. Our pipeline consists of two main stages: (1) converting SMILES strings into molecular graphs, and (2) applying graph neural network (GNN) architectures to learn from these graphs and predict compound activity. We outline each step in detail, from graph construction to model training and evaluation.

### 2.3.1 SMILES to Molecular Graph Conversion

To utilise graph neural networks for molecular property prediction, we first convert each molecule represented as a SMILES string into a graph structure. Each molecule is represented as a graph $G = (V, E)$, where $V$ is the set of atoms (nodes) and $E$ is the set of chemical bonds (edges) between them. The following method of conversion is proposed by Oxford Protein Informatics Group.[1]

We use RDKit to parse each SMILES string into a molecular object. For each atom, we extract a feature vector comprising: atom type (e.g., C, O, N; one-hot encoded), number of heavy-atom neighbors, formal charge, hybridization state, aromaticity, ring membership, scaled atomic mass, van der Waals radius, covalent radius, chirality and hydrogen count. These features are concatenated into a numerical vector for each atom, resulting in a node feature matrix $\mathbf{X} \in R^{n \times f}$, where $n$ is the number of atoms and $f$ is the feature dimension.

For each bond, we extract features including: bond type (single, double, triple, aromatic), conjugation status, ring membership, and stereochemistry (e.g., E/Z isomerism). These form the edge feature matrix $\mathbf{E}_f \in R^{m \times e}$, where $m$ is the number of edges (bi-directional) and $e$ is the feature dimension.

Graph connectivity is captured using an edge index matrix $\mathbf{E}_{\text{idx}} \in N^{2 \times m}$, which stores source and target atom indices for each edge.

Each graph, along with its node features, edge features, and corresponding label, is wrapped in a `torch_geometric.data.Data` object. All such graphs are collected into a list to be used for training and evaluating graph neural network models.

### 2.3.2 Graph Visualization

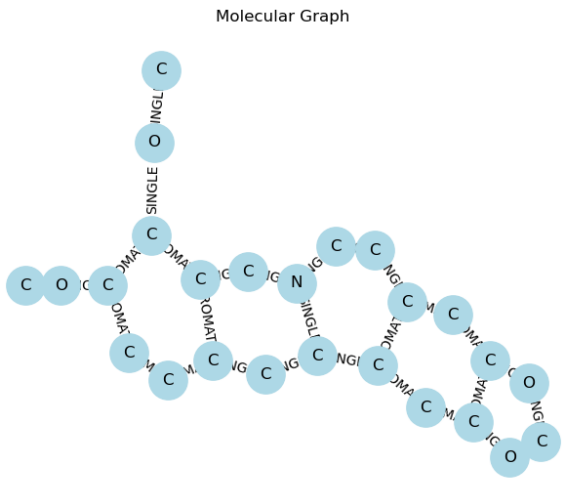


Figure 1: Molecular graph visualization showing atom types and bond types.

Figure 1 shows a sample molecular graph generated from a SMILES string. In this graph, each node represents an atom and is labeled by its atomic symbol (e.g., C, O, N), while each edge represents a chemical bond and is labeled by its bond type (e.g., single, aromatic). This visualization illustrates the topological structure that is passed to the graph neural network for learning.

Although this diagram displays only the atom and bond types, additional chemical properties such as chirality, hybridization state, formal charge, atomic radius, and stereochemistry are stored in the

node and edge feature matrices within the graph object. These features are essential for training and allow the model to capture complex chemical patterns.

### 2.3.3 Graph Neural Network Architectures: GCN and MPNN

To predict molecular interactions with 12 biological targets from the Tox21 dataset, we implement Graph Neural Network (GNN) architectures that learn from graph-structured molecular representations. Two GCN-based classification strategies are explored, alongside a more expressive variant known as the Message Passing Neural Network (MPNN):

- **Single-task GCN (st-GCN)**: A separate model is trained for each target, resulting in 12 individual binary classifiers.
- **Multi-task GCN (mt-GCN)**: A single shared model simultaneously predicts interaction probabilities for all 12 targets.
- **Message Passing Neural Network (MPNN)**: A graph neural architecture that enhances edge feature utilization through edge-conditioned weight generation.

**Single-task GCN (st-GCN) Architecture** The architecture for the single-task GCN is illustrated in Figure 2. It consists of three stacked Graph Convolutional layers (GCNConv) with ReLU activations, followed by a global mean pooling operation to aggregate node-level representations into a fixed-size graph-level vector. This is followed by a dropout layer for regularization and two fully connected layers. The final output is a two-dimensional vector passed through a softmax function, representing probabilities for the binary classification: interaction vs. no interaction.

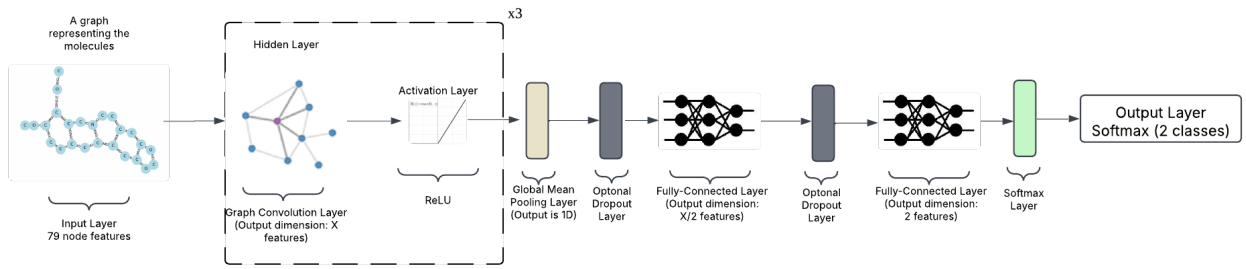


Figure 2: Single-task GCN architecture for binary classification.

**Multi-task GCN (st-GCN) Architecture** The multi-task GCN, shown in Figure 3, shares the same initial architecture as the single-task model, including the GCN layers, pooling, and fully connected layers. The key difference lies in the output layer: instead of a softmax for binary classification, the model uses a sigmoid activation function to produce 12 independent probabilities—one for each molecular target. This formulation allows the model to handle the multi-label nature of the problem. Correspondingly, the loss function is adjusted to use binary cross-entropy across all outputs, as detailed later in the report.

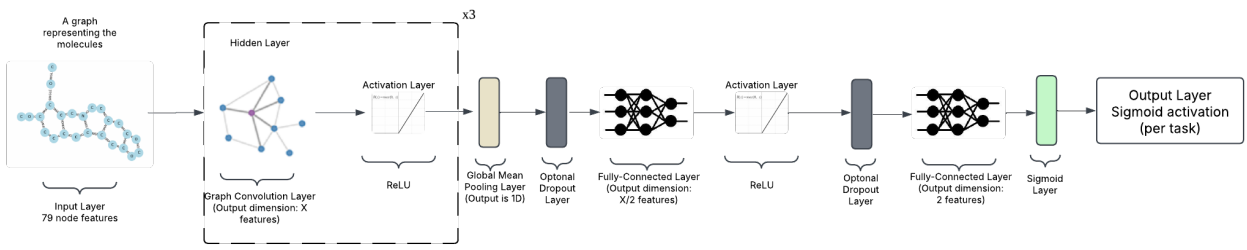


Figure 3: Multi-task GCN architecture for simultaneous prediction of 12 binary labels.

**Message Passing Neural Network (MPNN) Architecture** The Message Passing Neural Network (MPNN) is an alternative to the Graph Convolutional Network (GCN) that explicitly incorporates edge features into the learning process. As illustrated in Figure 4, MPNN replaces the fixed convolutional weights used in standard GCNs with edge-conditioned message functions. These are dynamically generated based on bond features, which are passed through a learned multi-layer perceptron (MLP). The output of the MLP is reshaped into a weight matrix and used to transform neighboring node features during message passing.[2] These transformed features are then aggregated using mean pooling to update the central node's representation.

In this study, the MPNN architecture is applied in a **multi-label classification setting**, analogous to the multi-task GCN, where the model simultaneously predicts activity across all 12 toxicity endpoints. Due to the increased computational complexity and higher parameter count of MPNN layers, we employ a simplified architecture consisting of a single `NNConv` layer, in contrast to the three-layer configuration used in the GCN models. This is followed by global mean pooling, dropout, and two fully connected layers. As with the multi-task GCN, the final output layer applies sigmoid activation functions to generate independent probability scores for each target.

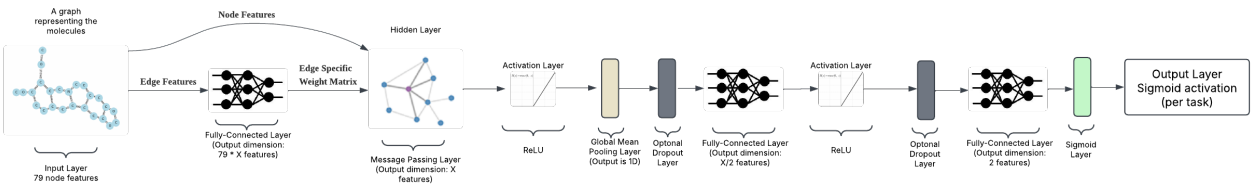


Figure 4: Message Passing Neural Network (MPNN) architecture for multi-target prediction.

### 2.3.4 Layer Descriptions

This section provides a concise explanation of each layer used in the GCN and MPNN architectures, describing its role and contribution to the final output.

**Input Layer:** The molecular structure, initially represented as a SMILES string, is converted into a molecular graph as described in the previous section. Each atom becomes a node with feature vectors, and each bond becomes an edge with edge-specific attributes. This graph representation forms the foundation for learning with graph neural networks.

**Graph Convolutional Layer:** This layer updates the representation of each node by aggregating feature information from its neighboring nodes and, optionally, edge attributes. In GCNs, this is performed using fixed weight sharing, while in MPNNs, the edge attributes are passed through an MLP to produce edge-specific weight matrices. These layers enable the model to learn spatial and relational patterns in molecular graphs.

**ReLU Activation Layer:** The Rectified Linear Unit (ReLU) is a non-linear activation function applied after each graph convolution and selected fully connected layers. ReLU introduces non-linearity into the model, enabling it to capture complex relationships between atomic features. It is also computationally efficient and helps mitigate the vanishing gradient problem, making it a popular choice in deep learning architectures.

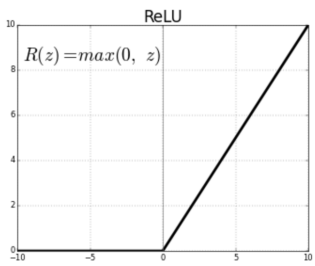


Figure 5: The ReLU equation and activation curve[4]

**Global Mean Pooling Layer:** While GNN layers produce node-level embeddings, a graph-level representation is required for classification tasks. Global mean pooling aggregates all node embeddings by computing their mean, resulting in a fixed-size vector that summarizes the entire molecule.

**Dropout Layer:** To improve generalisation and prevent overfitting, dropout is optionally applied after the pooling layer and the first fully connected layer. During training, dropout randomly sets a fraction of input units to zero, encouraging the network to learn more robust features and reduce overfitting.

**Fully Connected Layers:** Two fully connected (dense) layers are used to transform the graph-level embedding into the final prediction. The first dense layer reduces the feature dimensionality and is followed by a ReLU activation. The second layer maps the reduced representation to the final output dimension (either 2 for single-task or 12 for multi-task classification).

**Output Layer (Softmax or Sigmoid):** The final layer produces interpretable probabilities:

- In **single-task classification**, a softmax activation is applied over two output units to estimate the probability of molecular interaction (class 1) or no interaction (class 0).
- In **multi-task classification**, a sigmoid function is applied independently to each of the 12 outputs, each corresponding to a separate binary classification task.

These probabilities are compared against ground-truth labels using appropriate loss functions.

#### 2.3.5 Training Configuration and Setup

This section outlines the loss functions, training duration (epochs), and hyperparameter tuning process used for training both single-task and multi-task GNN models.

**Loss Functions** For the **single-task GCN**, we use the *Negative Log-Likelihood (NLL) Loss*. This is appropriate because the model produces two log-probability scores via a softmax function, corresponding to the binary classification of interaction versus non-interaction. NLL Loss is ideal when the target label is a class index and the model outputs log-probabilities.

In the case of **multi-task classification**, each target is treated as an independent binary label. We therefore apply the *Binary Cross-Entropy (BCE) Loss with Logits*, which combines a sigmoid activation with binary cross-entropy computation. Since the multi-task setup may include missing labels (i.e., `NaN` values), we apply a masking mechanism to exclude these entries during loss computation, ensuring numerical stability and preventing propagation of invalid gradients.

**Epoch Selection** To determine the number of training epochs, we performed exploratory runs while monitoring training loss, validation loss, and validation AUC scores. For the single-task GCNs, performance typically plateaued around 6 epochs, reflecting the relatively smaller model size and the focused learning objective of a single target. In contrast, the multi-task models required approximately 60 epochs to converge. This is expected, as multi-task learning involves optimizing multiple objectives simultaneously, which increases the complexity of the learning process and requires more iterations to achieve stable, shared feature representations across all 12 targets.

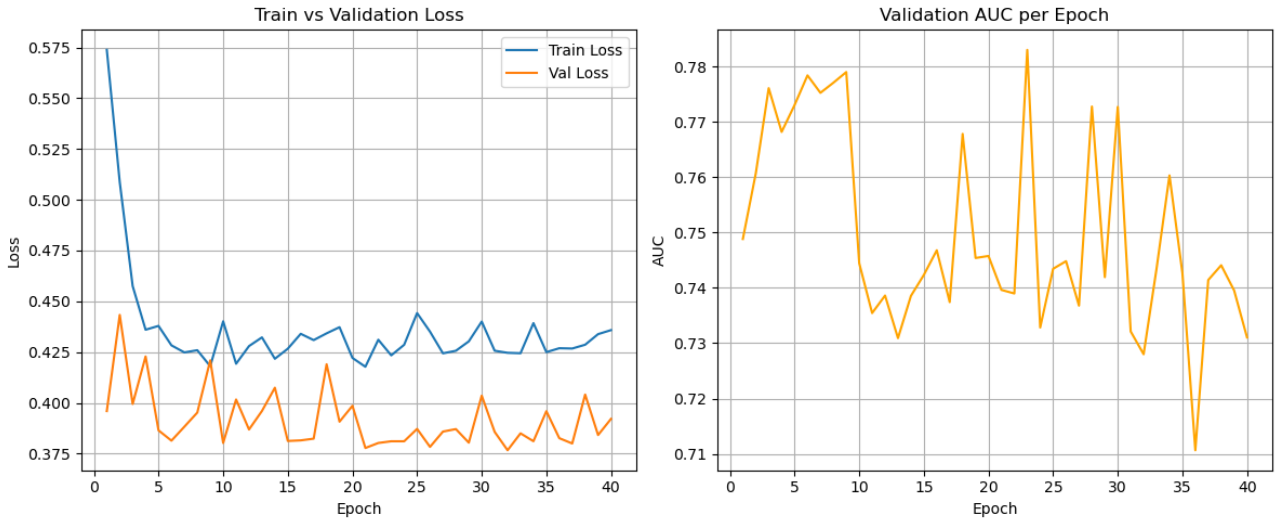


Figure 6: Training and validation loss curves with corresponding AUC scores during epoch progression (single-task example).

**Early Stopping Consideration** Although early stopping is a common practice for avoiding overfitting, it was avoided in this study. Given the significant class imbalance in the dataset, especially for the minority positive class, important learning patterns often emerge later in training. Early stopping based on validation loss could terminate training prematurely, before the model learns meaningful representations for minority classes. Additionally, allocating a separate validation set for early stopping would further reduce the amount of data available for training, a critical factor given the moderate size of the dataset.

**Hyperparameter Tuning** To optimise model performance, grid search was performed across several hyperparameters including hidden layer size, learning rate, dropout location, and dropout rate. The optimal combination was selected based on the highest macro-average AUC across 5-fold cross-validation. Detailed tuning results and selected configurations are presented in the following section.

# 3 Experiments

## 3.1 Hyperparameter Tuning for Tree-based Architecture

We conducted hyperparameter optimization using `RandomizedSearchCV` with 5-fold cross-validation to identify the best configurations for each model. Tables 2 and 3 summarize the search spaces defined for Random Forest, XGBoost, Logistic Regression, and Support Vector Machine (SVM).

Each grid was selected based on standard heuristics and prior work in binary classification, and was evaluated using the ROC-AUC score as the objective function. For all models, the best set of parameters was used to retrain on the full training set before final evaluation on the test set.

## 3.2 Graph-Based Architecture and Hyperparameter Tuning

To ensure a fair and consistent comparison with the classical machine learning models, we applied hyperparameter tuning to all graph-based architectures. A grid search strategy was employed in conjunction with 5-fold cross-validation to systematically explore combinations of key hyperparameters such as hidden layer dimensions, learning rate, and dropout configuration. The hyperparameters are presented in Table 4. This process aimed to optimise model performance while maintaining methodological consistency across all evaluated architectures.

The selected hyperparameters are intended to control model complexity, learning stability, and regularization:

Table 2: Hyperparameter tuning grids for Random Forest and XGBoost.

| **Random Forest** | **XGBoost** |
|---|---|
| • `n_estimators`: {100, 200, 500}<br>• `max_depth`: {5, 10, 20, None}<br>• `min_samples_split`: {2, 5, 10}<br>• `min_samples_leaf`: {1, 2, 4}<br>• `bootstrap`: {True, False} | • `n_estimators`: {100, 200, 300}<br>• `max_depth`: {3, 5, 7}<br>• `learning_rate`: {0.01, 0.1, 0.2}<br>• `subsample`: {0.6, 0.8, 1.0}<br>• `colsample_bytree`: {0.6, 0.8, 1.0}<br>• `gamma`: {0, 0.1, 0.3}<br>• `reg_alpha`: {0, 0.01, 0.1}<br>• `reg_lambda`: {1, 1.5, 2} |

Table 3: Hyperparameter tuning grids for Logistic Regression and SVM.

| **Logistic Regression** | **Support Vector Machine (SVM)** |
|---|---|
| • `penalty`: {`'l1'`, `'l2'`}<br>• `C`: {0.01, 0.1, 1, 10, 100}<br>• `solver`: {`'liblinear'`, `'saga'`} | • `C`: {0.1, 1, 10, 100}<br>• `kernel`: {`'linear'`, `'rbf'`}<br>• `gamma`: {`'scale'`, `0.001, 0.01, 0.1`} |

Table 4: Hyperparameter grid used for GCN and MPNN models.

| **GCN and MPNN Hyperparameters** |
|---|
| • `hidden_channels`: {64, 128, 256}<br>• `learning_rate`: {1e-3, 5e-4, 1e-4}<br>• `dropout_location`: {`'none'`, `'after_pool'`, `'after_fc1'`}<br>• `dropout_rate`: {0.2, 0.3} (only applied if `dropout_location` $\neq$ `'none'`) |

- **Hidden Channels:** This controls the dimensionality of the learned node embeddings, with larger values allowing the model to capture more complex patterns at the cost of increased computational resources and potential overfitting.
- **Learning Rate:** The initial step size in parameter updates during training. A higher learning rate speeds up convergence but may destabilize training, whereas a smaller rate results in slower but more stable learning.
- **Dropout Location and Rate:** Dropout is a regularization technique that randomly zeroes out neurons during training to reduce overfitting. We explore its effect at two different points in the network—after the pooling layer and after the first fully connected layer. Dropout is not applied when `dropout_location` is set to `'none'`.

Due to computational constraints—particularly the long training times of GNNs—this tuning process focused on a targeted subset of impactful hyperparameters. Additional tuning dimensions such as the number of layers or activation functions were not explored in this study to ensure feasibility.

# 4 Results and Discussion

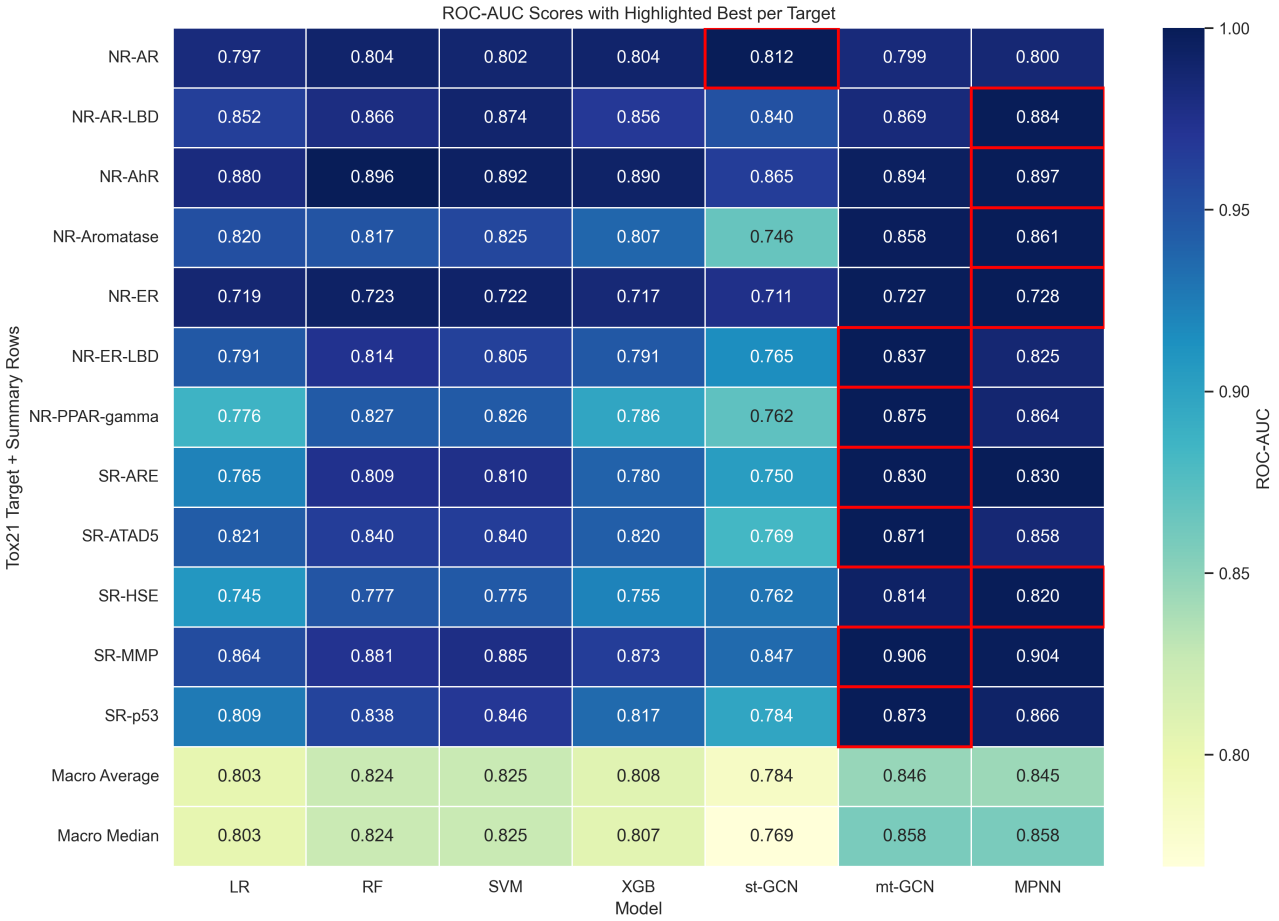


Figure 7: Heatmap Showing ROC-AUC Performance of Models on Tox21 Targets with Top Models Highlighted per Target

## 4.1 Model Performance Across Tox21 Targets

To analyze classification performance across specific toxicity endpoints, we computed the ROC-AUC scores of seven models—Logistic Regression, Random Forest, Support Vector Machine (SVM), XGBoost, single-task GCN (st-GCN), multi-task GCN (mt-GCN), and Message Passing Neural Network (MPNN)—on the twelve biological targets in the Tox21 dataset. The results are visualized in Figure 7 as a row-normalized heatmap, which displays both the exact AUC values and relative performance differences within each task, making it easier to identify the best-performing model per target.

To complement this view, Figure 8 provides a grouped bar chart that shows the raw ROC-AUC scores across all models for each individual target. This alternative visualization enables more intuitive side-by-side comparisons across models on a per-target basis and helps highlight consistent trends as well as task-specific variations that may be less apparent in heatmap formats.

The ROC-AUC metric was chosen due to its robustness in evaluating model performance in imbalanced datasets. Unlike accuracy or F1-score, which are sensitive to class distributions and thresholding, ROC-AUC measures the model's ability to rank positive instances higher than negative ones over all possible thresholds. This property makes it particularly well-suited for datasets like Tox21, where active compounds are often underrepresented. As discussed by Hanley and McNeil,[3] ROC-AUC remains invariant to class proportions and offers a comprehensive view of classifier discrimination performance.

**Logistic Regression (LR)** Logistic Regression (LR) consistently underperformed across nearly all Tox21 targets. Its ROC-AUC scores ranged from **0.719 (NR-ER)** to **0.880 (NR-AhR)**, with a **macro-average of 0.803** — the lowest among all models. Its linear structure inherently limits its

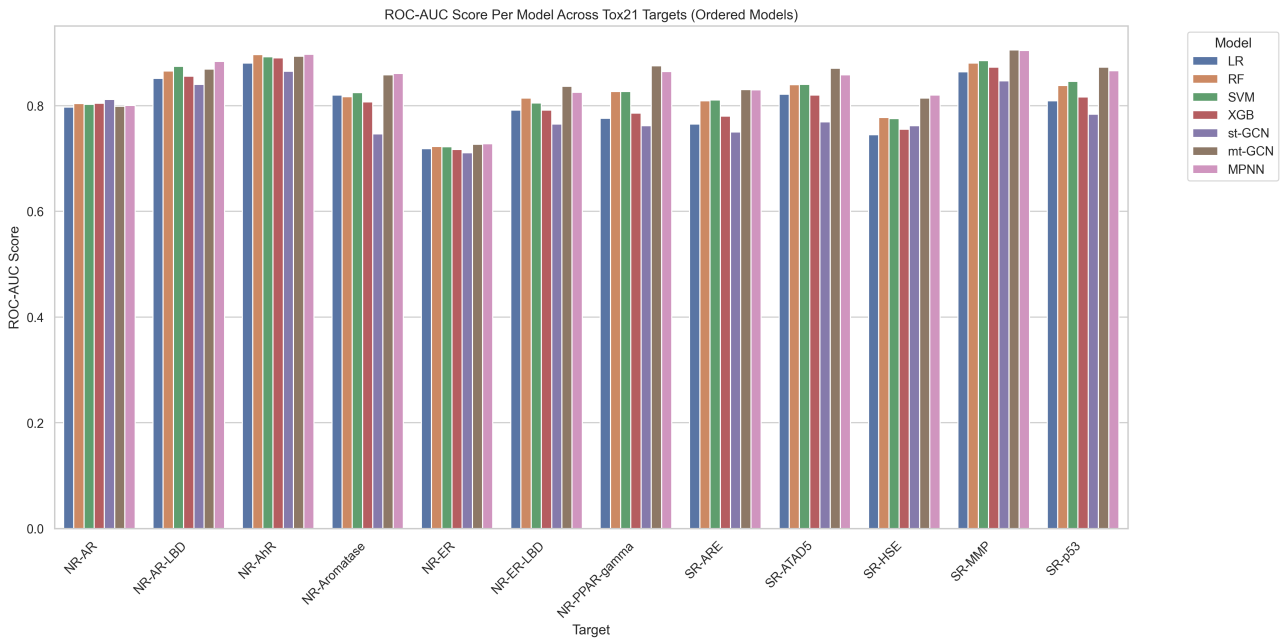


Figure 8: Chart of ROC-AUC scores for each model across the 12 Tox21 targets. Darker shades indicate stronger classification performance.

capacity to capture the complex and high-dimensional interactions present in molecular data. This reinforces the notion that linear baselines are insufficient for the nuanced task of toxicity prediction.

**Random Forest (RF)** Random Forest (RF) delivered solid performance on various tasks — notably **0.896 on NR-AhR** and **0.881 on SR-MMP** — and achieved a **macro-average ROC-AUC of 0.824**. However, the heatmap clearly shows that **RF did not obtain the highest score on any individual target**, as indicated by the absence of red-highlighted cells. This suggests that while RF is robust and effective, it is consistently outperformed by GNN-based models on a per-task basis, likely due to its inability to directly model molecular graph structures.

**Support Vector Machine (SVM)** SVM showed comparable strength to RF, with high scores such as **0.885 on SR-MMP** and **0.874 on NR-AR-LBD**, and a **macro-average ROC-AUC of 0.825**. Nevertheless, **SVM also did not lead on any individual target**, again confirming that traditional machine learning models lack the representational depth of GNNs. Its use of an RBF kernel enables it to model non-linear boundaries, but this still falls short of the end-to-end feature learning achieved by GNNs.

**XGBoost (XGB)** XGBoost produced consistently competitive results, such as **0.891 (NR-AhR)** and **0.873 (SR-MMP)**, and yielded a **macro-average ROC-AUC of 0.808**. However, it **did not achieve top performance on any task**, which suggests that while XGBoost's boosting framework promotes stability, it lacks the inductive bias necessary to exploit graph structures inherent to chemical compounds.

**Single-Task Graph Convolutional Network (st-GCN)** The st-GCN model exhibited the weakest performance among all GNN-based models, and even underperformed several classical baselines. With a **macro-average ROC-AUC of 0.784**, it only manages to secured a top score on one task (NR-AR). This can be attributed to its inability to leverage data from other tasks, resulting in insufficient supervision and limited graph feature learning. The relatively small dataset (∼5,000 samples per task) likely exacerbates this issue, as single-task GCNs require substantial data to generalize effectively.

**Multi-Task Graph Convolutional Network (mt-GCN)** mt-GCN emerged as the **most successful model**, achieving the **highest ROC-AUC in 6 out of 12 tasks**, clearly marked in red in the heatmap. It also secured the **highest macro-average ROC-AUC of 0.846**. By jointly learning across all targets, mt-GCN benefits from *shared representations* and increased effective sample size. This allows the model to retain data points with missing labels and enhance generalization through cross-task supervision. Its dominance underscores the value of multi-task learning in low-data, high-complexity molecular tasks.

**Message Passing Neural Network (MPNN)** MPNN also demonstrated excellent performance, achieving the **highest score in 5 out of 12 targets** — including **NR-AR-LBD (0.884)** and **NR-AhR (0.897)** — and attaining a **macro-average ROC-AUC of 0.845**, just a hair below mt-GCN. Its use of edge-conditioned message functions allows it to model task-specific molecular interactions effectively. The slightly lower average performance may be due to the use of only a single message-passing layer, limiting its ability to learn higher-order patterns. Still, the model's success on specific targets suggests it could benefit significantly from deeper architectures or pretraining.

**Comparative Summary** Only the **GNN-based models** (st-GCN, mt-GCN, MPNN) achieved task-leading ROC-AUC scores in the Tox21 benchmark. In particular:

- **mt-GCN** led in 6 targets
- **MPNN** led in 5 targets
- **st-GCN** led in 1 targets
- **Traditional ML models (LR, RF, SVM, XGB)** led in none

These results emphasize the critical advantage of *end-to-end graph representation learning*, especially when combined with multi-task strategies. While classical models like RF and SVM offer stability and interpretability, they consistently fall short when compared to GNNs in both per-task and average performance.

**Practical Implications and Future Work** Although deep learning models like mt-GCN and MPNN outperform traditional baselines, they are not without limitations. The dataset's relatively small size imposes challenges for generalization, particularly for single-task models. Future improvements may include:

- Deeper GNN architectures (e.g., multi-layer MPNNs)
- Pretraining on large molecular graphs
- Data augmentation or semi-supervised learning
- Incorporating attention mechanisms for interpretability

In practical settings, trade-offs between **performance and explainability** must also be considered. While mt-GCN sets a new benchmark, models like RF remain valuable where model transparency is paramount.

#### 4.1.1 Target-Level Trends

Among the 12 toxicity endpoints, NR-AhR consistently yields the highest ROC-AUC scores across all models, suggesting that this target benefits from strong signal quality or more distinguishable molecular patterns. In contrast, endpoints such as NR-ER and SR-HSE consistently exhibit lower performance, regardless of model architecture. This persistent disparity points to limitations not

in modeling strategy, but likely in data quality—such as label noise, insufficient structure-activity correlation, or under-representative features in the molecular graphs.

While multi-task models do generally outperform single-task counterparts across most targets, they do not fully resolve performance gaps for the weakest endpoints. This implies that shared learning helps to some extent but cannot compensate for fundamental issues in feature representation or target labeling. Future work may benefit from investigating the integrity of the activity labels for these underperforming targets, as well as incorporating more chemically meaningful features—such as stereochemistry, 3D conformation, or domain-specific graph augmentations, that could enhance the ability of models to distinguish between active and inactive compounds.

#### 4.1.2 Hyperparameter Configurations for Best Graph-Based Architectures

As shown in Figure 7, the single-task GCN (st-GCN) achieved the best performance on NR-AR, the multi-task GCN (mt-GCN) on six targets including NR-ER-LBD, NR-PPAR-gamma, SR-ARE, SR-ATAD5, SR-MMP, and SR-p53, while the MPNN outperformed other models on the remaining five targets: NR-AR-LBD, NR-AhR, NR-Aromatase, NR-ER, and SR-HSE.

Table 5 summarizes the optimal hyperparameter configurations for each graph-based model, as determined through grid search with 5-fold cross-validation.

Table 5: Best hyperparameter configurations

| **Model** | **Best Hyperparameters** |
|---|---|
| st-GCN | `hidden_channels`: 256, `lr`: 0.0005, `dropout_location`: 'none', `dropout_rate`: 0.0 |
| mt-GCN | `hidden_channels`: 256, `lr`: 0.001, `dropout_location`: 'after_pool', `dropout_rate`: 0.2 |
| MPNN | `hidden_channels`: 256, `lr`: 0.001, `dropout_location`: 'after_pool', `dropout_rate`: 0.2 |

## 5 Conclusion

This study presents a comprehensive evaluation of classical machine learning models and graph-based deep learning approaches on the Tox21 dataset, a benchmark for predicting compound toxicity across 12 biological targets. Our findings clearly indicate that graph neural networks (GNNs), especially the multi-task GCN (mt-GCN), are best suited for this domain.

The mt-GCN model achieved the highest performance, leading in 6 out of 12 tasks and attaining the best macro-average ROC-AUC score among all models. The message-passing neural network (MPNN) followed closely, particularly excelling on targets like NR-AhR and NR-AR-LBD. These results highlight the advantages of using graph-based molecular representations and multi-task learning in handling the sparsity and label incompleteness inherent to Tox21.

In contrast, traditional models such as Random Forest, SVM, and XGBoost showed stable yet suboptimal performance. While these models are effective in domains with rich tabular features, they lack the capacity to model the structural complexity of chemical compounds. Notably, none of the classical models outperformed the GNNs on any individual task, reinforcing the need for structure-aware methods in molecular machine learning.

Despite the strong results from GNNs, several limitations remain. The relatively small sample size per task (approximately 5,000 compounds) restricts deep models' ability to generalize, particularly in single-task settings. Additionally, interpretability and computational cost remain concerns when deploying these models in practical settings.

Future work could address these limitations by exploring deeper GNN architectures, incorporating semi-supervised learning or data augmentation, and pretraining on larger chemical graph datasets. Moreover, enhancing interpretability through attention mechanisms or post hoc explanation techniques will be vital for adoption in high-stakes toxicology applications.

In conclusion, this study reaffirms the suitability of GNN-based approaches for toxicity prediction and demonstrates the value of multi-task learning in low-data, multi-label biomedical domains such as Tox21. These advancements have implications for accelerating early-stage drug discovery, reducing reliance on animal testing, and improving chemical safety assessment pipelines.

## References


[1] Joe Davies. How to turn a smiles string into a molecular graph for pytorch geometric, February 2022. Accessed: 2025-04-25. URL: `https://www.blopig.com/blog/2022/02/how-to-turn-a-smiles-string-into-a-molecular-graph-for-pytorch-geometric/`.

[2] Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70, pages 1263–1272. PMLR, 2017.

[3] James A Hanley and Barbara J McNeil. The meaning and use of the area under a receiver operating characteristic (roc) curve. *Radiology*, 143(1):29–36, 1982.

[4] Prateek Vishnu. Activation functions in neural networks, 2020. Accessed: 2025-04-25. URL: `https://prateekvishnu.medium.com/activation-functions-in-neural-networks-bf5c542d5fec`.